Interfaz de línea de comandos

Automatización, control y eficiencia desde la terminal



Objetivos de aprendizaje

1

Introducción a la consola

Comprender la interfaz de línea de comandos en diferentes sistemas operativos.

2

Comandos básicos

Ejecutar comandos fundamentales y aprender técnicas de manipulación de archivos.

3

Scripts sencillos

Crear scripts básicos para automatizar tareas y operaciones comunes.

4

CMD vs BASH

Comparar los entornos de línea de comandos entre Windows (CMD) y los sistemas basados en Unix (BASH).

¿Qué es la interfaz de línea de comandos (CLI)?

La interfaz de línea de comandos es un método basado en texto para interactuar con el sistema operativo de tu computadora. En lugar de hacer clic en botones y navegar por los menús, escribes comandos que le indican a la computadora que realice operaciones específicas.

Interfaz basada en texto

Interactúa con el sistema a través de comandos escritos en lugar de elementos gráficos.

Alternativa a la GUI

Proporciona acceso directo a las funciones del sistema que pueden estar ocultas o anidadas en interfaces gráficas.

Potente y eficiente

Permite la ejecución rápida de tareas complejas y la automatización de operaciones repetitivas.



Entornos comunes de línea de comandos

Windows



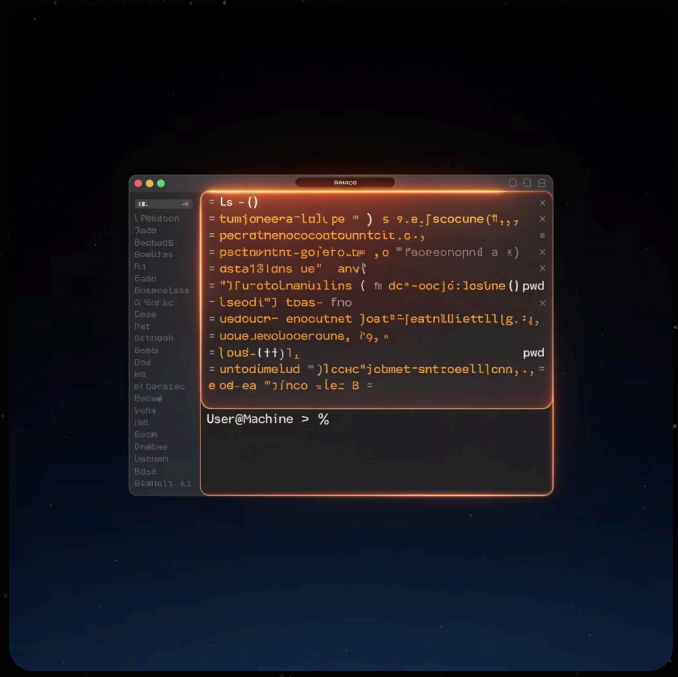
- Símbolo del sistema (CMD)
- PowerShell (alternativa más potente)
- Terminal de Windows (aplicación de consola moderna)

Linux



- Terminal + Bash (Bourne Again SHell)
- Múltiples emuladores de terminal disponibles
- Varias shells: Bash, Zsh, Fish

macOS



- Aplicación Terminal
- Usa Bash (versiones anteriores) o Zsh (más recientes)
- iTerm2 (alternativa popular)

Bash vs CMD: Una comparación

Característica	Bash (Linux/macOS)	CMD (Windows)
Potencia	Alta	Limitada
Scripts	archivos .sh	archivos .bat
Soporte	Muy extenso	Básico
Uso actual	Entornos profesionales	Sistemas básicos / heredados
Alternativas	Zsh, Fish	PowerShell
Distinción entre mayúsculas y minúsculas	Sí	No
Utilidades integradas	Muchas herramientas potentes	Menos opciones

PowerShell cierra la brecha al ofrecer funciones avanzadas en Windows que son más comparables a Bash.

Comandos comunes de archivos y directorios

Acción	Linux/macOS (Bash)	Windows (CMD)
Ver contenidos	ls	dir
Cambiar directorio	cd	cd
Crear carpeta	mkdir folder	mkdir folder
Crear archivo vacío	touch file.txt	type nul > file.txt
Copiar	cp a.txt b.txt	copy a.txt b.txt
Mover	mv a.txt folder/	move a.txt folder\
Borrar archivo	rm a.txt	del a.txt
Borrar carpeta	rm -r folder	rmdir /s folder

Nota: Las opciones de comando (flags) pueden variar entre sistemas.

Comandos útiles adicionales

Visualización del contenido del archivo

Bash: cat, less, head, tail

Ejemplos:

```
cat file.txt # Mostrar archivo completo
less file.txt # Vista paginada
head -n 5 file.txt # Primeras 5 líneas
tail -f log.txt # Seguir actualizaciones de archivos
```

CMD: type, more

```
type file.txt # Mostrar archivo completo
more file.txt # Vista paginada
```

Búsqueda de texto

Bash: grep

```
grep "pattern" file.txt
grep -r "pattern" folder/
```

CMD: findstr

```
findstr "pattern" file.txt
findstr /s "pattern" folder\*
```

Gestión de procesos

Bash: ps, top, kill

```
ps aux # Lista todos los procesos
top # Vista de proceso en vivo
kill 1234 # Matar proceso por ID
```

CMD: tasklist, taskkill

```
tasklist # Lista todos los procesos
taskkill /PID 1234 # Matar proceso por ID
```

¿Qué es un script?



Un script es un archivo de texto que contiene una secuencia de comandos que puede ser ejecutada por el intérprete de línea de comandos. Los scripts te permiten:

- Almacenar una serie de comandos en un solo archivo
- Ejecutar múltiples comandos con una sola acción
- Automatizar tareas repetitivas
- Crear herramientas y utilidades personalizadas
- Realizar operaciones condicionales
- Procesar archivos y datos en lotes

Los scripts ahorran tiempo y reducen el error humano al automatizar procesos manuales, haciéndolos esenciales para la administración y el desarrollo de sistemas.

Estructura básica de un script

Ejemplo de script Bash

```
#!/bin/bash
# Este es un comentario
echo "Hola, bienvenido $USER"
echo "Hoy es $(date)"
echo "Tu directorio actual es:"
pwd
```

Guarda como **greeting.sh** y hazlo ejecutable:

```
chmod +x greeting.sh
./greeting.sh
```

Ejemplo de archivo por lotes CMD

```
@echo off
REM Este es un comentario
echo Hola, %USERNAME%
echo Hoy es %DATE%
echo Tu directorio actual es:
cd
```

Guarda como **greeting.bat** y ejecútalo:

```
greeting.bat
```

La línea shebang (`#!/bin/bash`) en los scripts de Bash le dice al sistema qué intérprete usar. Los archivos por lotes CMD no necesitan esto.

Flujo de control: Condicionales y bucles

Condicionales de Bash

```
if [ -f file.txt ]; then
    echo "File exists"
elif [ -d folder ]; then
    echo "Folder exists"
else
    echo "Neither exists"
fi
```

Condicionales de CMD

```
IF EXIST file.txt (
    echo File exists
) ELSE IF EXIST folder (
    echo Folder exists
) ELSE (
    echo Neither exists
)
```

Bucles de Bash

```
for i in {1..5}; do
    echo "Iteration $i"
done

while [ $counter -lt 5 ]; do
    echo "Counter: $counter"
    counter=$((counter+1))
done
```

Bucles de CMD

```
FOR /L %%i IN (1,1,5) DO (
 echo Iteration %%i
)

SET counter=0
:loop
IF %counter% LSS 5 (
 echo Counter: %counter%
 SET /A counter+=1
 GOTO loop
)
```

Ejemplos sencillos de scripts de copia de seguridad

Script de copia de seguridad de Bash

```
#!/bin/bash
# Script de copia de seguridad sencillo

# Obtener la fecha actual en formato AAAA-MM-DD
DATE=$(date +%F)

# Directorios de origen y destino
SRC="/home/user/documents"
DEST="/home/user/backups/backup_$DATE"

# Crear copia de seguridad
echo "Creando copia de seguridad de $SRC a $DEST"
mkdir -p $DEST
cp -r $SRC/* $DEST

echo "¡Copia de seguridad completada!"
```

Script de copia de seguridad de CMD

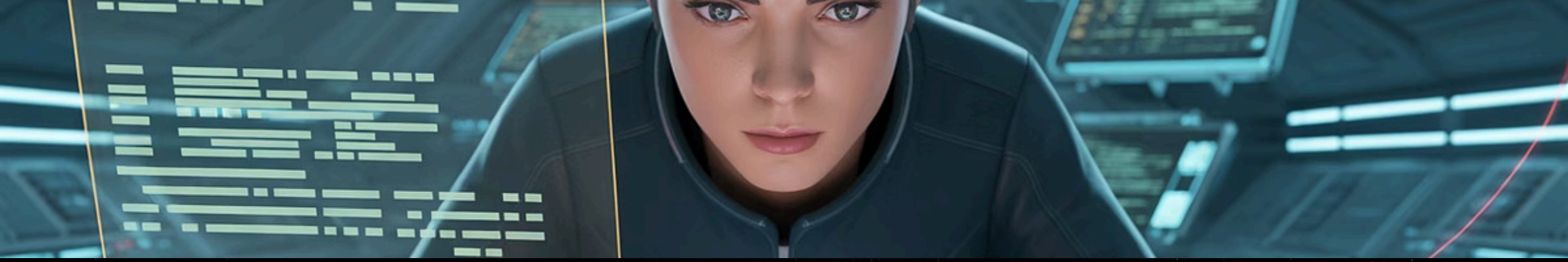
```
@echo off
REM Script de copia de seguridad sencillo

REM Obtener la fecha actual en formato AAAA-MM-DD
SET DATE=%date:~6,4%-%date:~3,2%-%date:~0,2%

REM Directorios de origen y destino
SET SRC=C:\Users\user\Documents
SET DEST=C:\Backups\Backup_%DATE%

REM Crear copia de seguridad
echo Creando copia de seguridad de %SRC% a %DEST%
mkdir %DEST%
xcopy %SRC%\* %DEST% /E /I /Y

echo ¡Copia de seguridad completada!
```



Ejercicio práctico: Creación de un analizador de registros

Tarea: Crea un script que analice los archivos de registro e informe de los errores

Sigue estos pasos para crear tu propio analizador de registros:

1. Abre tu terminal o símbolo del sistema
2. Crea un nuevo archivo de script (loganalyzer.sh o loganalyzer.bat)
3. Escribe código para buscar "ERROR" en un archivo de registro y contar las apariciones
4. Añade funcionalidad para extraer y mostrar los últimos 5 errores
5. Ejecuta tu script en un archivo de registro de muestra

Este ejercicio te ayudará a practicar la manipulación de archivos, la búsqueda de texto y el formato de salida, habilidades esenciales para la administración de sistemas.

Aplicaciones del mundo real de la CLI



Administración del sistema

Configuración del servidor, administración de usuarios, supervisión de procesos y tareas de mantenimiento automatizadas



Desarrollo de software

Control de versiones con Git, compilación de código, ejecución de pruebas e implementación de aplicaciones



Procesamiento de datos

Procesamiento de texto, extracción de datos, operaciones de archivos por lotes e informes automatizados



Ciberseguridad

Análisis de redes, análisis de registros, auditoría de seguridad y detección de intrusiones

Variables de entorno y configuración

Variables de entorno de Bash

Las variables de entorno almacenan información utilizada por el sistema y las aplicaciones.

```
# Ver todas las variables de entorno
env

# Ver variable específica
echo $HOME
echo $PATH

# Establecer variable temporal
export MYVAR="Hola Mundo"

# Hacer permanente (agregar a ~/.bashrc)
echo 'export MYVAR="Hola Mundo"' >> ~/.bashrc
```

Variables de entorno de CMD

Windows utiliza conceptos similares pero con una sintaxis diferente.

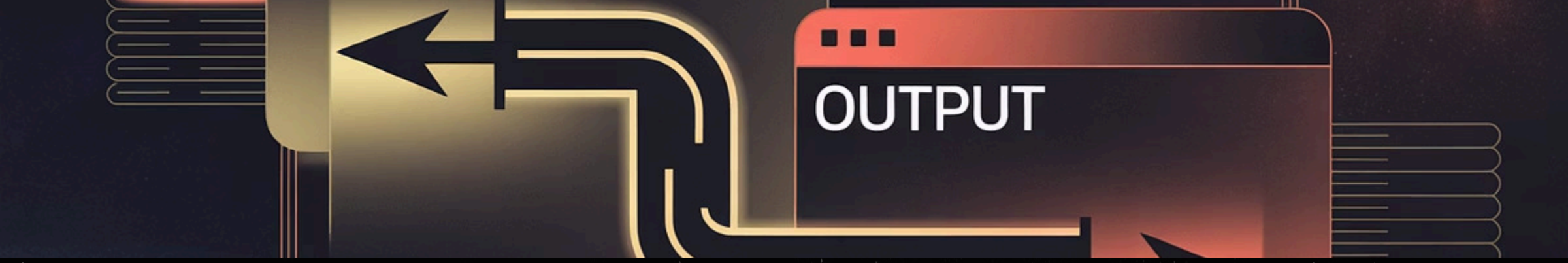
```
REM Ver todas las variables de entorno
set

REM Ver variable específica
echo %USERPROFILE%
echo %PATH%

REM Establecer variable temporal
set MYVAR=Hola Mundo

REM Hacer permanente
setx MYVAR "Hola Mundo"
```

Las variables de entorno controlan el comportamiento del programa y proporcionan información del sistema accesible a los scripts.



Redirección de entrada/salida

Controlar de dónde provienen y hacia dónde van los datos

Salida estándar (stdout)

```
ls > filelist.txt     # Guardar la salida en un archivo
ls >> filelist.txt    # Añadir al archivo
dir > filelist.txt    # Equivalente en Windows
```

Entrada estándar (stdin)

```
sort < unsorted.txt   # Leer desde un archivo
mail -s "Log" user < log.txt
```

Error estándar (stderr)

```
command 2> errors.txt  # Guardar errores
command 2>&1 > all.txt  # Tanto stdout como stderr
```

Tuberías

```
ls -la | grep ".txt" # Filtrar la salida
cat file.txt | wc -l # Contar líneas
```

Atajos y trucos de la línea de comandos

Atajos de Bash

- **Tab**: Autocompletar comandos y nombres de archivo
- **Ctrl+R**: Buscar en el historial de comandos
- **Ctrl+A/E**: Mover al principio/final de la línea
- **Ctrl+U**: Borrar la línea antes del cursor
- **!!:** Repetir el último comando
- **comando &**: Ejecutar en segundo plano
- ***** y **?**: Comodines para la coincidencia de patrones

Atajos de CMD

- **Tab**: Ciclo a través de nombres de archivos/carpetas
- **F7**: Mostrar el historial de comandos
- **Flechas arriba/abajo**: Navegar por el historial
- **Alt+F7**: Borrar el historial de comandos
- **F3**: Repetir el último comando
- **start command**: Ejecutar en una nueva ventana
- ***** y **?**: Comodines para la coincidencia de patrones

Dominar estos atajos aumenta significativamente tu eficiencia al trabajar con la línea de comandos.

Consideraciones de seguridad de la línea de comandos

Comprenda los comandos antes de la ejecución

Siempre investigue los comandos con los que no esté familiarizado antes de ejecutarlos, especialmente aquellos que se encuentran en línea. Los comandos maliciosos pueden causar graves daños a su sistema.

Tenga cuidado con los comandos peligrosos

Algunos comandos pueden ser destructivos si se usan incorrectamente:

- `rm -rf /` (Linux/macOS): Elimina todo
- `format C:` (Windows): Formatea la unidad de su sistema
- `:(){ :|:& };:` (Bash): Bomba de bifurcación que bloquea los sistemas

Tenga cuidado con los privilegios elevados

Los comandos que se ejecutan con sudo (Linux/macOS) o como Administrador (Windows) tienen acceso sin restricciones a su sistema. Use privilegios elevados solo cuando sea necesario.

Valide los scripts antes de ejecutarlos

Siempre revise los scripts descargados de Internet antes de la ejecución, o use entornos de espacio aislado para probar código desconocido.

Mejores prácticas para el uso de la línea de comandos



- ## Comandos de prueba en entornos seguros

  Ejecute primero los comandos potencialmente destructivos en los directorios de prueba.

- ## Comente sus scripts

  Use # (Bash) o REM (CMD) para documentar lo que hacen sus scripts.

- ## Use rutas absolutas para operaciones críticas

  Evite las rutas relativas en los scripts que realizan tareas importantes del sistema.

- ## Cree copias de seguridad antes de realizar cambios importantes

  Siempre haga una copia de seguridad de los archivos importantes antes de modificarlos con scripts.

- ## Use el control de versiones para los scripts

  Almacene los scripts en repositorios de Git para rastrear los cambios y colaborar.

- ## Incorpore el manejo de errores en los scripts

  Incluya comprobaciones y mensajes de error para facilitar la depuración.

Ejercicios prácticos

Aplica lo que has aprendido con estas actividades prácticas:

1

Navegación por el sistema de archivos

Abre tu terminal y practica:

- Crear una estructura de directorios (al menos 3 niveles de profundidad)
- Crear archivos en diferentes directorios
- Mover y copiar archivos entre directorios
- Listar archivos con diferentes opciones (ocultos, detalles, etc.)

2

Script de saludo simple

Crea un script que:

- Muestra un mensaje de bienvenida con tu nombre de usuario
- Muestra la fecha y hora actuales
- Lista el contenido de tu directorio actual
- Muestra información del sistema (versión del sistema operativo, uso de memoria)

3

Herramienta de copia de seguridad de archivos

Escribe un script que:

- Crea una carpeta de copia de seguridad con marca de tiempo
- Copia archivos de un directorio de origen a la copia de seguridad
- Cuenta y muestra el número de archivos de los que se ha realizado una copia de seguridad
- Lista los archivos modificados recientemente de las últimas 24 horas

Recursos para seguir aprendiendo

Documentación en línea

- **Manual de Bash:** Documentación de GNU Bash
- **CMD de Windows:** Referencia de la línea de comandos de Microsoft
- **Páginas de manual:** Documentación integrada (comando man)
- **Páginas TLDR:** Ejemplos de comandos simplificados

Libros

- "The Linux Command Line" de William Shotts
- "Wicked Cool Shell Scripts" de Dave Taylor
- "PowerShell in Action" de Bruce Payette

Aprendizaje interactivo

- **Codecademy:** Aprende la línea de comandos
- **Linux Journey:** Tutoriales interactivos de Linux
- **Over The Wire - Bandit:** Juego de seguridad de la línea de comandos
- **ShellCheck:** Herramienta de análisis de scripts

Comunidades

- Stack Overflow
- Unix & Linux Stack Exchange
- Reddit: r/bash, r/PowerShell
- GitHub: Repositorios de scripts

Recuerda: ¡La mejor manera de aprender la línea de comandos es mediante la práctica constante en tareas del mundo real!